# Frontend Intern Task

| | |
|---|---|
| Owner | Ganesh Jayachandran |
| Verification | Empty |
| Tags | Empty |
| Last edited time | December 28, 2025 5:47 PM |

Hide 2 properties

## ReactFlow Canvas

Complete this take-home task **within the given time period** shared with you. Focus on correctness, clarity, and clean architecture over extra features.

### Objective

Build a small, responsive "App Graph Builder" UI that matches the provided screenshot layout and demonstrates:

- Layout composition (top bar, left rail, right app panel, dotted canvas)
- ReactFlow (xyflow) basics (3 nodes + interactions)
- A service node inspector UI (tabs + controls)

- A service node inspector UI (tabs + controls)
- TanStack Query data fetching (mock APIs)
- Zustand state management (selected app/node + UI state)
- Strict TypeScript + linting + scripts

### Canvas Layout

## Tech Stack (must use)

- **React + Vite**
- **TypeScript (strict: true)**
- **ReactFlow (xyflow)**
- **shadcn/ui**
- **TanStack Query**
- **Zustand**
- **Mock API calls** (no real backend)

## What to Build

### 1) Screenshot Layout

Recreate the layout structure (not pixel-perfect, but should be clearly aligned with the screenshot):

- **Top bar** (brand/title + a couple of placeholder actions)
- **Left rail** (icon-style nav; static is fine)
- **Right panel** split into:
  - **App selector / Apps list**
  - **Node Inspector** (shows when a node is selected)
- **Center canvas** with a **dotted background** and ReactFlow inside

**Responsive requirement:**

- On small screens, the right panel should become a **slide-over drawer** (open/close controlled by Zustand).

---

### 2) ReactFlow (xyflow) Basics

Render a graph with at least **3 nodes** and **2 edges**.

Required interactions:

- Drag nodes
- Select a node (click)
- Delete a selected node with **Delete/Backspace**

- Delete a selected node with **Delete/Backspace**
- Zoom + pan (default ReactFlow behavior is okay)
- Fit view on initial load (or provide a "Fit" button in top bar)

Canvas styling:

- Use **dotted background** (ReactFlow Background variant dots) or a CSS dotted pattern.

---

## 3) Service Node Inspector UI (right panel)

When a node is selected, show a "Service Node" inspector with:

**A) Status pill**

- A small pill/badge that reflects node status (example statuses: Healthy, Degraded, Down).

**B) Tabs**

- Minimum **2 tabs** (example: Config and Runtime).

**C) Slider + numeric input (synced)**

- A slider (0–100 is fine)
- A numeric input
- They must stay in sync both ways.
- Persist the value to the selected node's data.

**D) Basic fields**

- Node name (editable input)

- Optional: description textarea

Use **shadcn/ui** for inputs, tabs, badge, buttons, etc.

---

## 4) TanStack Query (mock APIs)

Implement these mock endpoints (in-memory, simulated latency):

- GET /apps → returns a list of apps
- GET /apps/:appId/graph → returns nodes + edges for the selected app

Required behavior:

- Loading state (skeleton/spinner is fine)
- Error state (simulate an error with a toggle or random failure)
- Cached results via TanStack Query
- When app changes, graph refetches and renders

**Mocking approach (choose one):**

- Simple setTimeout wrapper returning Promises
- MSW (Mock Service Worker)

---

## 5) Zustand (required state)

Use Zustand for non-server UI/app state:

- selectedAppId
- selectedNodeId

- selectedAppId
- selectedNodeId
- isMobilePanelOpen
- activeInspectorTab

Expectations:

- State should be minimal and well-structured
- Derived data should not be over-stored (prefer selectors)

---

## Functional Requirements Summary (checklist)

- Layout: top bar, left rail, right panel, dotted canvas
- Responsive: right panel becomes a mobile drawer
- ReactFlow: 3 nodes, drag, select, delete, zoom/pan
- Node inspector: tabs + status pill + synced slider/input
- TanStack Query: mock /apps and /apps/:appId/graph + loading/error
- Zustand: selected app/node, mobile panel open, active tab
- TypeScript strict + linting + scripts

### Mock API calls

Using Mock Service Worker (MSW) or a Vite plugin like `vite-plugin-mock-dev-server`

### Mock API calls

Using Mock Service Worker (MSW) or a Vite plugin like `vite-plugin-mock-dev-server`

```
// /src/mocks/handlers.js
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('/api/user', () => {
    return HttpResponse.json({
      code: 0,
      data: { name: 'Mock User', role: 'admin' },
    });
  }),
  // Add other handlers for POST, PUT, etc.
];
```

---

## Engineering Requirements

### TypeScript + Linting

- tsconfig must enable **strict** mode
- ESLint configured for React + TS
- Prettier (optional but recommended)

- Prettier (optional but recommended)

### Scripts (required)

Provide at least these scripts:

- dev
- build
- preview
- lint
- typecheck

### Code quality expectations

- Components split cleanly (layout, canvas, inspector, data hooks)
- Avoid prop drilling where Zustand fits better
- Keep ReactFlow state updates predictable

---

## Deliverables

Submit a GitHub repository containing:

- Source code
- README.md with:
  - Setup instructions
  - Key decisions (brief)

## Deliverables

Submit a GitHub repository containing:

- Source code
- README.md with:
  - Setup instructions
  - Key decisions (brief)
  - Known limitations

Optional but helpful:

- deploy to Vercel or Cloudflare Pages

## Evaluation Criteria

We will evaluate:

- Correctness of required features
- UI structure & responsiveness
- ReactFlow integration quality (state, selection, deletion)
- Clean TanStack Query usage (loading/error/caching)
- Zustand state correctness (minimal but sufficient)

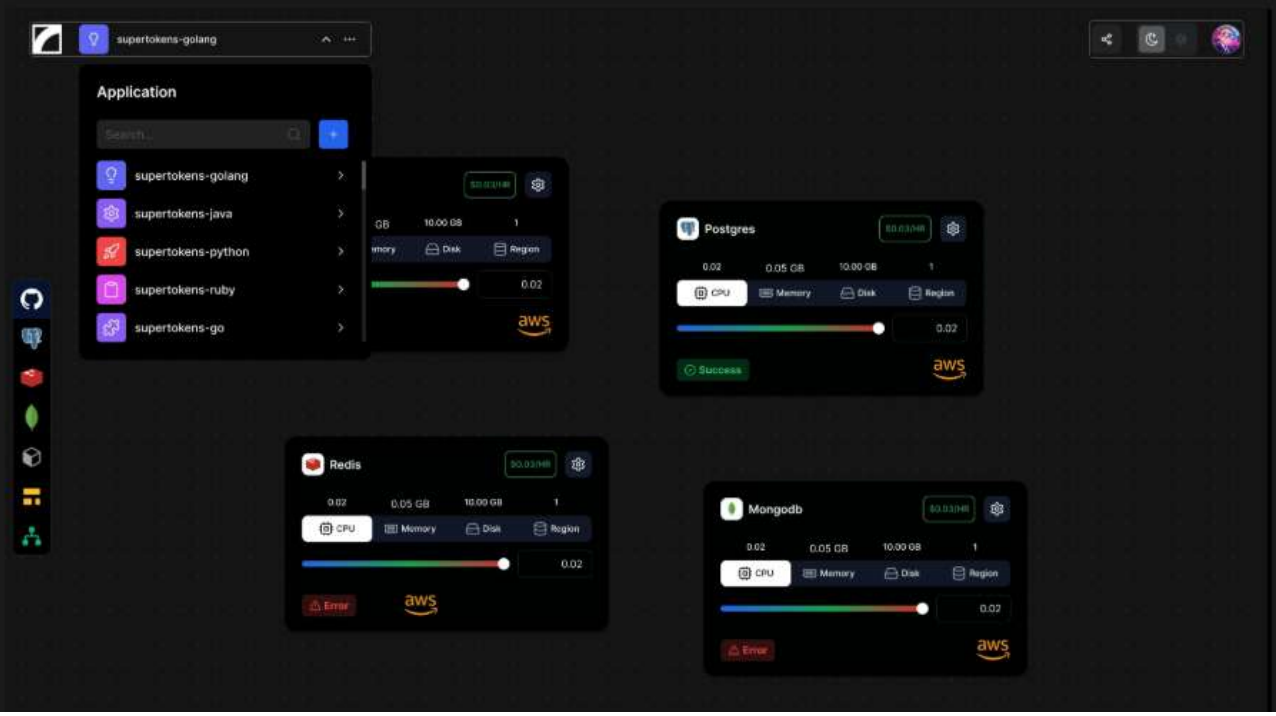
- UI structure & responsiveness
- ReactFlow integration quality (state, selection, deletion)
- Clean TanStack Query usage (loading/error/caching)
- Zustand state correctness (minimal but sufficient)
- Code readability, TypeScript strictness, and tooling quality

---

## Bonus (only if time permits)

- Add "Add Node" button (creates a new service node)
- Node types (Service vs DB) with different styling
- Persist inspector edits into ReactFlow node data cleanly
- Keyboard shortcuts (Fit view, toggle panel)

Canvas Page


supertokens-golang
Application
Search...
supertokens-golang
supertokens-java
supertokens-python
supertokens-ruby
supertokens-go
Postgres
0.02
0.05 GB
10.00 GB
1
CPU
Memory
Disk
Region
0.02
Success
aws
Redis
Error
Mongodb